Clase 332 — Agentes de IA y el Model Context Protocol (MCP) para seguridad

> Parte: **18 — IA aplicada a la ciberseguridad** · Fuente: Model Context Protocol (modelcontextprotocol.io) · OWASP Top 10 for LLM ⏱ Duración estimada: **100 min** · Nivel: **Intermedio**

🎯 Objetivo

Entender qué es un **agente de IA** y cómo el **Model Context Protocol (MCP)** le permite usar herramientas reales (escáneres, bases de datos, sistemas de archivos) de forma estandarizada. Es la pieza que convierte un chatbot que "explica" en un asistente que "hace" — con todo lo que eso implica en poder y en riesgo para un flujo de seguridad.

📚 Resultados de aprendizaje

Al finalizar, el alumno podrá:

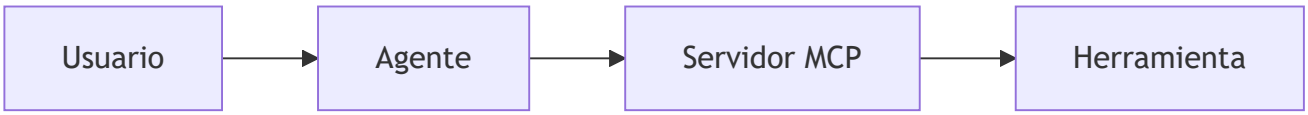
1. **Diferenciar** un LLM conversacional de un **agente** que ejecuta acciones en un bucle.
2. **Describir** la arquitectura MCP: cliente (agente), servidor MCP y herramientas expuestas.
3. **Explicar** por qué MCP estandariza la conexión IA↔herramientas (como "USB para herramientas de IA").
4. **Identificar** los riesgos de seguridad de dar herramientas a un agente (acción no supervisada, prompt injection, permisos excesivos).
5. **Diseñar** un modelo de permisos mínimo para un agente con herramientas.

🗺 Temas

#	Tema	Por qué importa
1	De chatbot a agente	El agente actúa; hay que gobernar esas acciones.
2	Bucle percibir–razonar–actuar	Cómo un agente encadena herramientas hasta un objetivo.
3	Arquitectura MCP	Cliente–servidor–herramientas; transporte y mensajes.
4	Tools, resources y prompts	Los tres tipos de capacidad que expone un servidor MCP.
5	Riesgos del agente con herramientas	Acción destructiva, prompt injection, exfiltración.
6	Permisos y aprobación	Mínimo privilegio y aprobación humana de acciones sensibles.

🧠 Explicación en profundidad

Un agente combina modelo, estado y herramientas. MCP estandariza intercambio de contexto y capacidades, pero no convierte un servidor en confiable. Cliente, servidor, recurso y herramienta forman fronteras separadas.



El diagrama separa lenguaje de autoridad. El modelo puede proponer; la política determina si la operación pertenece al alcance; la herramienta produce evidencia; y una persona o control determinista decide transiciones de alto impacto. Se registran prompt relevante, parámetros validados, versión, salida, aprobador y cleanup sin almacenar secretos innecesarios.

Caso razonado

Un servidor describe una herramienta como lectura, pero ejecuta cambios. La política se basa en capacidad y prueba, no en descripción.

📱 Glosario operativo

Término	Definición
Capability	Operación concreta que un componente ofrece bajo permisos.
Guardrail	Capa que reduce una capacidad o detecta su uso; no garantía absoluta.
Audit trail	Registro ordenado de solicitudes, decisiones, ejecuciones y resultados.

✅ Criterio de dominio

El alumno demuestra dominio cuando reproduce el flujo completo, puede atribuir cada acción, evita que texto no confiable otorgue autoridad y explica qué control contiene un fallo del modelo.

📖 Definiciones y características

Agente de IA : LLM que, en un bucle, decide qué herramienta usar, la invoca, observa el resultado y continúa hasta cumplir un objetivo. Característica clave: **actúa** sobre el mundo.

MCP (Model Context Protocol) : Protocolo abierto que estandariza cómo un cliente de IA se conecta a servidores que exponen herramientas, recursos y prompts. Evita integraciones a medida para cada herramienta.

Servidor MCP : Proceso que expone capacidades (p. ej. "ejecutar nmap", "leer un archivo") a través del protocolo, para que cualquier cliente compatible las use.

Tool (herramienta) : Acción que el agente puede invocar (con parámetros) y cuyo resultado vuelve al modelo. En seguridad, una tool puede lanzar un escaneo o consultar una base de datos.

Prompt injection (a un agente) : Contenido malicioso en los datos que el agente procesa (una web, un banner de servicio) que intenta secuestrar sus instrucciones. Riesgo central de los agentes con herramientas.

🧰 Herramientas y preparación

Un cliente de IA con soporte MCP (p. ej. Claude Code) y, opcionalmente, un servidor MCP de ejemplo. En la clase siguiente montaremos **kali-mcp**. Aquí basta con entender la arquitectura y experimentar con un servidor MCP sencillo si lo tienes disponible.

🧪 Laboratorio guiado

1. **Mapa mental.** Dibuja el flujo: agente → mensaje MCP → servidor → herramienta → resultado → agente. Marca dónde entra la aprobación humana.
2. **Inventario de tools.** Para un agente de pentest, lista 6 herramientas que expondrías y sus parámetros mínimos (p. ej. `scan(target, ports)`).
3. **Modelo de permisos.** Clasifica esas tools en: *auto* (solo lectura, p. ej. escaneo pasivo), *aprobación requerida* (escaneo activo), *prohibido a la IA* (acciones destructivas).
4. **Superficie de prompt injection.** Identifica qué datos "no confiables" podría leer el agente (banners, webs, respuestas de servicios) y cómo podrían manipularlo.
5. **Política.** Escribe 5 reglas para operar un agente con herramientas de forma segura.

✍️ Ejercicios

1. Explica con tus palabras la diferencia entre un LLM y un agente.
2. ¿Qué ventaja da MCP frente a integrar cada herramienta a mano?
3. Da un ejemplo concreto de prompt injection contra un agente de recon.
4. Diseña la regla de aprobación para una tool que ejecuta `sqlmap`.
5. ¿Por qué "mínimo privilegio" aplica también a las herramientas de un agente?

📝 Reto verificable

Diseña, en un diagrama + tabla, la arquitectura de un agente de pentest con MCP: tools expuestas, nivel de permiso de cada una y los puntos de aprobación humana.

Criterio de aceptación: toda acción con impacto (escaneo activo, explotación, escritura) requiere aprobación explícita; las de solo lectura pueden ser automáticas; justificas cada decisión.

⚠️ Errores comunes

Síntoma / mensaje	Causa y cómo arreglar
Dar al agente permisos totales "para que fluya"	Riesgo enorme. Aplica mínimo privilegio y aprobación por acción sensible.
Confiar en datos que el agente lee de la red	Pueden contener prompt injection. Trátalos como entrada no confiable.
No registrar qué hizo el agente	Sin trazabilidad no hay auditoría. Loguea cada tool-call y su resultado.
Exponer credenciales en el contexto del agente	Puede filtrarlas. Usa secretos gestionados, no en el prompt.
Asumir que el servidor MCP es de confianza	Un servidor MCP malicioso puede abusar del cliente. Usa solo servidores auditados.

❓ Preguntas frecuentes

❓ **¿MCP es solo de un proveedor?** No; es un protocolo abierto adoptado por varios clientes de IA. Por eso un mismo servidor MCP (como kali-mcp) puede usarse desde distintos agentes.

❓ **¿El agente ejecuta las herramientas directamente en mi máquina?** Depende del diseño. En kali-mcp, las herramientas corren dentro de un contenedor Docker de Kali, aislado, y el agente habla con un gateway — no ejecuta binarios arbitrarios en tu host.

❓ **¿Cuál es el mayor riesgo?** Dar acciones potentes sin supervisión y la prompt injection: que datos del objetivo manipulen al agente para que haga algo fuera de alcance.

> 🔗 **Conexión con el abuso de recursos de IA (LLMjacking) — Parte 18 · IA aplicada.** Un agente que opera herramientas de ciberseguridad consume inferencia con credenciales propias. Aplica aquí lo visto en las clases 295 y 297: clave tras un gateway, presupuesto por tarea y registro de invocaciones, y no entregues al agente claves de proveedor en un entorno donde ejecute código.

🔗 Referencias

- Model Context Protocol — especificación
- OWASP Top 10 for LLM Applications — LLM01 Prompt Injection, LLM06 Excessive Agency.
- kali-mcp (MIT) — caso de estudio de la clase siguiente.

📥 Material descargable

- 📄 Guía en PDF — versión imprimible de esta clase.
- 🎞️ Presentación (PPTX) — deck para proyectar en clase.

⬅️ Clase anterior

Clase 331 — IA generativa y LLMs en ciberseguridad: panorama, capacidades y límites

➡️ Siguiente clase

Clase 333 — kali-mcp: orquestar herramientas de Kali desde un agente de IA